

Department of Computer Engineering
University of Peradeniya
CO5420: Neural Networks and Deep Learning
Lab 02: Introduction to Perceptron

Objectives

- Understand and implement a single-layer perceptron using Python.
- Analyze the perceptron's behavior for simple classification problems.
- Observe the effects of input variations on perceptron performance.
- Interpret learning curves and decision boundaries generated during training.

Task 01: Manual vs. Library Implementation of a Perceptron

You will manually implement a single-layer perceptron and compare its performance with a ready-made implementation using sklearn.

Step 1: Define the Dataset

```
# Define input and output for the AND Gate
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])
```

Step 2: Manual Implementation of Perceptron

```
# Manual Implementation of Perceptron
import numpy as np

def step_function(x):
    return 1 if x >= 0 else 0

def perceptron_training(X, y, learning_rate=0.1, epochs=10):
    weights = np.zeros(X.shape[1])
    bias = 0

    for _ in range(epochs):
        for i in range(len(X)):
            activation = np.dot(X[i], weights) + bias
            prediction = step_function(activation)
            error = y[i] - prediction
            weights += learning_rate * error * X[i]
            bias += learning_rate * error
```

```

        return weights, bias

# Train manually
weights, bias = perceptron_training(X, y)
print("Manual Weights:", weights)
print("Manual Bias:", bias)

```

Step 3: Ready-Made Implementation using sklearn

```

from sklearn.linear_model import Perceptron

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X, y)

print("Sklearn Weights:", model.coef_)
print("Sklearn Bias:", model.intercept_)

```

Step 4: Compare Results

```

# Predictions
manual_predictions = [step_function(np.dot(x, weights) + bias) for x in X]
sklearn_predictions = model.predict(X)

print("Manual Predictions:", manual_predictions)
print("Sklearn Predictions:", sklearn_predictions)

```

Questions

- What are the similarities and differences between manual implementation and Sklearn's model?
- Why is it beneficial to use libraries like sklearn in practice?

Task 02: Logical Gate Classification

You will demonstrate the ability of a single-layer perceptron to model basic logic gates: AND, OR, and NOT.

Instructions

1. Define the input-output mapping for each logic gate.
 - Example: AND Gate
 - Inputs: [(0, 0), (0, 1), (1, 0), (1, 1)]
 - Outputs: [0, 0, 0, 1]
2. Implement the perceptron training step-by-step:

Step 1: Train the Perceptron

```
from sklearn.linear_model import Perceptron

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X, y)
```

Step 2: Evaluate and Visualize

```
from sklearn.metrics import accuracy_score

# Predictions
y_pred = model.predict(X)
print("Accuracy:", accuracy_score(y, y_pred))

# Decision Boundary Visualization
import matplotlib.pyplot as plt

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.01),
    np.arange(y_min, y_max, 0.01)
)

Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k')
plt.title("Decision Boundary")
plt.show()
```

Questions

- Why does a single-layer perceptron fail for certain logic gates and what are those logic gates?
- How does adjusting the learning rate affect the perceptron's performance?

Task 03: Effect of Outliers on Training

You will analyze the training process of a perceptron on a dataset with outliers.

Step 1: Generate Data

```
import numpy as np

# Generate two Gaussian distributions
class1 = np.random.multivariate_normal([2, 2], [[1, 0], [0, 1]], 100)
class2 = np.random.multivariate_normal([5, 5], [[1, 0], [0, 1]], 100)

# Add an outlier
class2 = np.vstack([class2, [10, 10]])

# Labels
X = np.vstack([class1, class2])
y = np.hstack([np.ones(100), -np.ones(101)])
```

Step 2: Train the Perceptron

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X_train, y_train)
```

Step 3: Evaluate

```
import matplotlib.pyplot as plt

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Questions

- How does the presence of outliers affect training time and accuracy?
- What strategies can be used to mitigate the effect of outliers?

Task 04: Real-World Classification

Scenario

Apply a single-layer perceptron to classify grayscale images of handwritten digits (e.g., '0' vs. '1').

Step 1: Load the Dataset

```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler

digits = load_digits()

mask = (digits.target == 0) | (digits.target == 1)
X = digits.data[mask]
y = digits.target[mask]

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Step 2: Train the Perceptron

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X_train, y_train)
```

Step 3: Evaluate Performance

```
from sklearn.metrics import confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Step 4: Visualize Confusion Matrix

```
import seaborn as sns

sns.heatmap(
    confusion_matrix(y_test, y_pred),
    annot=True,
    fmt='d',
```

```
cmap='Blues'  
)  
  
plt.title("Confusion Matrix")  
plt.xlabel("Predicted")  
plt.ylabel("True")  
plt.show()
```

Questions

- What changes would you make to improve accuracy?

Submission

Submit a ZIP file named `eyyxxx_lab02.zip` containing the following:

1. PDF Document:

- Implement each task step by step.
- Include answers to all questions specific to each task.
- Provide required plots for each task, including decision boundaries and confusion matrices.

2. Implementation Files:

- Python scripts (.py or .ipynb files) for all tasks.

Note: xxx indicates your registration number and yy mean your batch year in all cases. Those who have plagiarized content will be heavily penalized.